

编译原理大作业 2：语义分析与中间代码生成器

——类 Rust 语言的静态语义检查与四元式生成

2351078 李堂辉

2351837 高胜寒

2352197 冉子易

(按学号排序)

同济大学

2026 年 5 月 8 日

目录

项目概述

语义分析与符号表

中间代码生成 (IR)

系统架构

测试与演示

思考与总结

环境与执行

任务要求

大作业目标

在词法、语法分析基础上，完成静态语义检查与中间代码生成

基本功能

- 静态语义检查（作用域/类型）
- 控制流语法制导翻译
- 生成四元式指令

覆盖规则

- 精准拦截多类静态语义错误
- 生成线性中间代码 (IR)
- 实现符号表的嵌套作用域管理

技术选型

实现语言

Rust

- 类型安全, 内存安全
- 枚举类型天然适合 Token 和 AST
- 模式匹配简化分析逻辑

分析方法

- **语义检查**: 深度优先遍历, 符号表栈机制
- **代码生成**: 语法制导翻译与回填
- 四元式枚举封装, 方便扩展机器码生成

静态语义检查与符号表

核心机制

在遍历抽象语法树 (AST) 过程中, 维护作用域栈, 实现变量重影 (Shadowing) 与生命周期管理。

多级符号表结构

- 使用 `Vec<HashMap>` 模拟栈结构
- 进入 Block/If/While 时
 `push_scope`
- 退出时 `pop_scope`

拦截的静态语义错误

- 使用未声明变量
- 不可变变量被二次赋值
- 类型不一致 (赋值、加减运算)
- 越界的 `break / continue`

四元式 (Quadruple) 格式

定义

基于三地址码思想, 定义了简洁抽象的四元式系统:

(操作符 (Op), 源操作数 1(Arg1), 源操作数 2(Arg2), 结果 (Result))

操作数 (Arg) 类型

- Empty (-): 无操作数
- Int(i64): 字面量常量
- Var(String): 变量名或分配的地址
- Label(String): 跳转标签 (如 L0)
- Temp(usize): 编译器临时变量 (如 t1)

控制流与内存访问指令

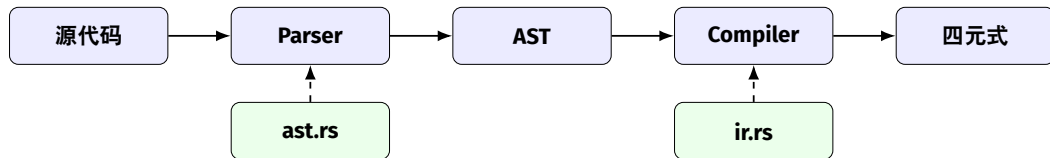
- JUMP_IF_FALSE: 条件不成立跳转
- JUMP: 无条件跳转
- Deref / REF: 解引用与取地址
- LOAD_ARRAY: 数组元素的读取

控制流生成：以 while 循环为例

```
1 let start_label = self.new_label();
2 let end_label = self.new_label();
3
4 self.ir.emit(Op::Label, Arg::Empty, Arg::Empty, Arg::Label(start_label));
5
6 let (_, cond_arg) = self.compile_expression(cond);
7 self.ir.emit(Op::JumpIfFalse, cond_arg, Arg::Empty, Arg::Label(end_label));
8
9 self.loop_stack.push((start_label.clone(), end_label.clone(), None));
10 self.compile_block(body);
11 self.loop_stack.pop();
12
13 self.ir.emit(Op::Jump, Arg::Empty, Arg::Empty, Arg::Label(start_label));
14 self.ir.emit(Op::Label, Arg::Empty, Arg::Empty, Arg::Label(end_label));
15
```

控制流回填 (Backpatching): 利用标号和 loop_stack 完成 break 和 continue 的指令地址重定向。

模块架构



- **token.rs**: Token 类型定义
- **lexer.rs**: 词法扫描器

- **ast.rs**: AST 节点 + 打印
- **parser.rs**: 递归下降分析器

诊断语义错误

我们设计了包含多种静态语义违规的源程序进行拦截测试。

```
1 fn test_type_mismatch() {  
2     let mut a: i32 = 1;  
3     let mut b: [i32; 3] = [1, 2, 3];  
4     a = b;  
5 }  
6 fn test_immutable_assign() {  
7     let a: i32 = 1;  
8     a = 2;  
9 }
```

语义报错诊断输出:

```
[Semantic Error] 赋值类型不一致: 左值 I32, 右值 Array(I32, 3)  
[Semantic Error] 不可变变量 'a' 不允许被二次赋值  
[Semantic Error] 二元运算类型不匹配: I32 + Tuple([I32, I32])  
[Semantic Error] break 必须出现在循环体内  
[Semantic Error] continue 必须出现在循环体内
```

四元式生成：表达式与赋值

```
1 fn program_7_2(x:i32, y:i32) -> i32 {  
2   let mut t:i32 = x*x+x;  
3   t = t*x*y;  
4   return t;  
5 }
```

对应的 IR 输出:

program_7_2:

1:	(	PARAM	,	-	,	-	,	x	)
2:	(	PARAM	,	-	,	-	,	y	)
3:	(	MUL	,	x	,	x	,	t21	)
4:	(	ADD	,	t21	,	x	,	t22	)
5:	(	ASSIGN	,	t22	,	-	,	t	)
6:	(	MUL	,	x	,	y	,	t23	)
7:	(	ADD	,	t	,	t23	,	t24	)
8:	(	ASSIGN	,	t24	,	-	,	t	)
9:	(	RETURN	,	t	,	-	,	-	)

四元式生成：循环跳转指令

```
1 fn program_5_1(mut n:i32) {  
2     while n>0 {  
3         n=n-1;  
4     }  
5 }
```

对应的 IR 输出：

program_5_1:

1: (PARAM , - , - , n)

L27:

3: (GT , n , 0 , t13)

4: (JUMP_IF_FALSE, t13 , - , L28)

5: (SUB , n , 1 , t14)

6: (ASSIGN , t14 , - , n)

7: (JUMP , - , - , L27)

L28:

9: (RETURN , - , - , -)

测试结果与指标

代码生成全覆盖

原 28 个正常测试程序均成功通过静态语义检查，并输出正确的四元式指令序列！

统计信息

- 语义检查：对非法 break、类型错误、未声明变量可直接拦截
- 四元式：精准生成了条件跳转 (JUMP_IF_FALSE)、算术指令
- 临时变量的 t_id 以及标号 L_id 被严格分配与管理

项目编译与执行

编译环境

- Rust 1.70+, Cargo
- LaTeX (XeLaTeX), latexmk (用于生成报告)

常用编译与执行命令

- `cargo build --release`: 编译生成 release 二进制文件
- `cargo run --release`: 运行内置语法树解析测试
- `cargo run --release -- tests/test_all.rs.txt`: 指定外部程序测试

总结

完成内容

- 在大作业 1 的框架上开发了**静态语义分析器**和**中间代码生成器**
- 利用符号表完美实现了同名变量在嵌套作用域中的精准遮蔽 (Shadowing)
- 通过基于栈的标号缓存, 实现了 while, break, continue 的跳转逻辑
- 全面拦截不安全操作 (赋值给不可变对象、类型不一致)
- 功能测试与语义测试样例全部满足预期行为

个人收获

- 深入理解了静态语义约束对保证编程语言安全性的重大意义
- 掌握了控制流如何通过标号和条件跳转拍平到线性的中间代码
- 体会到架构分离 (分析器与编译器独立) 极大提升了工程的可维护性

参考资料

- 陈火旺. 程序设计语言编译原理 (第 3 版). 国防工业出版社.
- Alfred V. Aho et al. Compilers: Principles, Techniques and Tools (Second Edition). 赵建华等译. 机械工业出版社.
- The Rust Reference & Rust By Example.

谢谢!